

DDPG-Based Adaptive Portfolio Allocation for a Five-Asset Equity Portfolio

Abstract—Portfolio allocation requires repeated decisions under changing market conditions, where changes in asset weights affect both portfolio exposure and transaction costs. This work presents a Deep Deterministic Policy Gradient (DDPG)-based framework for adaptive portfolio allocation across a fixed set of five U.S. technology equities: Apple, Microsoft, Alphabet, Amazon, and NVIDIA. Daily historical market data from 2015 to 2024 are used to construct a 50-dimensional state representation comprising nine technical features for each asset and the current portfolio weights. The DDPG Actor generates continuous portfolio allocations, while the Critic evaluates the corresponding state-action pairs. Portfolio turnover is explicitly incorporated through a transaction-cost model, and the data are divided chronologically into training, validation, and held-out test periods. The trained policy is evaluated on the unseen 2024 test period against an equal-weight portfolio using final portfolio value, total return, Sharpe ratio, maximum drawdown, annualized return, annualized volatility, and transaction costs. On the held-out test period, the DDPG strategy achieved a total return of 44.63%, while the equal-weight portfolio achieved 57.95%. The results demonstrate the application of DDPG to continuous portfolio allocation and provide an out-of-sample assessment of its allocation behavior under explicit transaction-cost considerations.

Index Terms—Deep Reinforcement Learning, Deep Deterministic Policy Gradient, Portfolio Allocation, Equity Markets, Continuous Action Space, Transaction Costs, Out-of-Sample Evaluation

I. INTRODUCTION

A. Background and Motivation

Portfolio management involves allocating capital among multiple financial assets while considering return and risk. Since financial market conditions change over time, an allocation that is suitable during one market period may not remain appropriate as new observations become available. This makes portfolio allocation a sequential decision-making problem in which the allocation can be updated according to the observed market state. Reinforcement learning (RL) provides a framework for learning such sequential decisions through interaction between an agent and an environment. Deep reinforcement learning further represents the decision policy and value function using neural networks, allowing the approach to handle higher-dimensional market representations. For portfolio allocation, continuous-action reinforcement learning is particularly relevant because the decision can be expressed directly as the proportion of capital assigned to each asset rather than as separate buy, sell, or hold actions. Deep Deterministic Policy Gradient (DDPG) is an actor-critic reinforcement learning method designed for continuous action spaces. In the proposed framework, the Actor generates the target portfolio allocation, while the Critic evaluates the quality of the resulting

state-action pair. The portfolio environment then applies the allocation and accounts for portfolio return and transaction costs. This work investigates DDPG-based adaptive allocation for a fixed portfolio of five U.S. technology equities: Apple (AAPL), Microsoft (MSFT), Alphabet (GOOGL), Amazon (AMZN), and NVIDIA (NVDA). Nine technical features are calculated for each asset, and the current portfolio weights are included in the agent state, resulting in a 50-dimensional state representation. Transaction costs are explicitly incorporated to account for the effect of portfolio turnover on the realized reward. The learned policy is evaluated using a chronological training, validation, and held-out testing procedure. The 2024 market period is reserved for out-of-sample evaluation, and the resulting portfolio is compared with an equal-weight allocation using return and risk measures. This provides a controlled evaluation of the learned allocation policy under the defined experimental configuration.

B. Problem Statement

The portfolio allocation problem considered in this work is formulated as a sequential continuous-control problem. At each trading step, the agent observes market information together with the current portfolio allocation and determines the target proportion of capital assigned to each of the five assets. The objective is to learn a DDPG-based allocation policy that adapts to changing market conditions while accounting for the transaction costs associated with portfolio turnover. The resulting allocation is applied to the portfolio environment, which calculates the portfolio return, transaction cost, and resulting reward before advancing to the next market state. The current portfolio weights are retained as part of the state so that subsequent decisions depend on both market conditions and the existing allocation. The central problem addressed in this study is to evaluate whether the learned DDPG policy can produce dynamic portfolio allocations under the defined transaction-cost setting. The policy is developed using chronological training and validation periods and subsequently evaluated on the held-out 2024 test period against an equal-weight portfolio using return, risk, and transaction-cost measures.

C. Research Gap

Existing research demonstrates the potential of reinforcement learning for portfolio management, but the experimental settings vary substantially in terms of asset classes, portfolio sizes, market periods, state representations, learning algorithms, reward formulations, and transaction-cost assumptions.

Previous studies have considered cryptocurrency markets, large equity portfolios, and market indexes, while more recent approaches have incorporated mechanisms such as modern portfolio theory, risk-sensitive rewards, and transaction-cost constraints [1], [2], [6], [7]. These variations make direct comparison difficult because portfolio performance can be influenced by both the learning method and the experimental configuration. Based on this observation, this work adopts a controlled configuration for examining DDPG-based adaptive portfolio allocation. The portfolio consists of five fixed U.S. technology equities: AAPL, MSFT, GOOGL, AMZN, and NVDA. Each asset is represented using nine predefined technical features, while the current portfolio weights are included in the state, resulting in a 50-dimensional representation. Continuous portfolio weights and an explicit transaction-cost model are used to represent the allocation problem. The study further applies a chronological training, validation, and held-out testing procedure, with the complete 2024 period reserved for out-of-sample evaluation. The resulting DDPG policy is compared with an equal-weight portfolio under the same test period, providing a controlled basis for examining its observed allocation behavior, return, risk, and transaction-cost characteristics.

D. Contributions

The main contributions of this work are as follows:

- A DDPG-based adaptive portfolio allocation framework is developed for a controlled portfolio of five U.S. technology equities: AAPL, MSFT, GOOGL, AMZN, and NVDA.
- A 50-dimensional state representation is defined using nine technical features for each asset and the current portfolio weights, providing both market and portfolio information to the agent.
- Portfolio turnover is explicitly incorporated through a transaction-cost model, allowing allocation changes to affect the portfolio reward.
- A chronological training, validation, and held-out testing protocol is used, with 2024 reserved for out-of-sample evaluation. The learned policy is evaluated against an equal-weight portfolio using return, risk, and transaction-cost measures.

II. RELATED WORK

A. Deep Reinforcement Learning for Portfolio Management

Deep reinforcement learning has been applied to portfolio management using different asset classes, portfolio sizes, and decision formulations. Jiang *et al.* developed a deep reinforcement learning framework using Ensemble of Identical Independent Evaluators, Portfolio-Vector Memory, and online stochastic batch learning, and evaluated CNN, RNN, and LSTM variants on cryptocurrency data with transaction commissions [1]. Liu *et al.* applied an actor-critic approach to a portfolio of 30 stocks and compared the resulting strategy with the Dow Jones Industrial Average and a minimum-variance portfolio using cumulative return and Sharpe ratio [2]. Yang

et al. combined PPO, A2C, and DDPG for automated stock trading using 30 Dow Jones stocks, illustrating the use of multiple deep reinforcement learning algorithms for portfolio decisions [3].

Betancourt and Chen addressed portfolio management with a changing number of assets and incorporated transaction-cost minimization into their formulation, evaluating the approach using cryptocurrency data [4]. Jang and Seong combined modern portfolio theory with DDPG for dynamic stock allocation and evaluated the resulting portfolios using measures including Sharpe ratio, annualized return, maximum drawdown, and final portfolio value [6]. Jiang *et al.* investigated TD3-based portfolio selection using risk and transaction-cost considerations with constituents of the Dow Jones Industrial Average and S&P 100 [7]. These studies demonstrate the relevance of continuous-control reinforcement learning to portfolio allocation while using substantially different experimental configurations.

B. Risk, Transaction Costs, and Evaluation

Risk and transaction costs are important considerations when evaluating reinforcement learning-based portfolio strategies. Shen and Li used financial indicators with DDPG and evaluated portfolio performance using cumulative portfolio value and Sharpe ratio [5]. Cheng and Sun developed a multi-agent reinforcement learning framework for multi-asset portfolio management and incorporated the Sortino ratio to account for downside risk [8]. Jiang *et al.* applied Q-learning to dynamic risk control and asset allocation using daily Yahoo Finance data and evaluated cumulative return, Sharpe ratio, and maximum drawdown [9]. Choudhary *et al.* proposed a risk-adjusted framework using multiple PPO agents trained with return, differential Sharpe ratio, and maximum drawdown objectives and evaluated the resulting strategies using several risk and return measures [10].

The reviewed studies use different asset universes, state representations, reward designs, transaction-cost settings, and evaluation procedures. Therefore, observed portfolio performance depends on both the learning approach and the experimental configuration. In this work, evaluation uses total return, Sharpe ratio, maximum drawdown, annualized return, annualized volatility, and transaction costs within a controlled five-asset equity setting.

III. DATASET AND EXPERIMENTAL SETUP

A. Data Source and Asset Selection

The experimental data consist of daily historical market observations obtained from Yahoo Finance using the Python `yfinance` package. The raw data include Open, High, Low, Close, Adjusted Close, and Volume values for five U.S. technology equities: Apple Inc. (AAPL), Microsoft Corporation (MSFT), Alphabet Inc. (GOOGL), Amazon.com Inc. (AMZN), and NVIDIA Corporation (NVDA). The selected assets provide a fixed multi-asset setting for evaluating dynamic portfolio allocation.

The historical data cover the period from January 1, 2015, to December 31, 2024. This period was selected to provide a sufficiently long sequence of daily observations containing different market conditions. Technical indicators were subsequently calculated from the historical observations. Because several indicators require a minimum number of preceding observations, the feature-engineered dataset begins on March 16, 2015.

The portfolio is initialized with an equal allocation across the five assets. With an initial portfolio value of \$100,000, each asset receives an initial weight of 20%. The portfolio remains long-only, and the allocation weights are constrained to be non-negative and normalized to sum to one.

B. Feature Engineering and State Representation

Nine market features are generated for each asset from the historical price data. These features represent price movement, trend information, momentum, and Bollinger Band characteristics. The feature set consists of daily return, 20-day simple moving average ratio, 50-day simple moving average ratio, normalized 14-day Relative Strength Index (RSI), normalized Moving Average Convergence Divergence (MACD), normalized MACD signal, normalized MACD histogram, Bollinger Band position, and Bollinger Band width.

For each of the five assets, the nine features form a 45-dimensional market representation. The current portfolio weights are then appended to this representation so that the agent receives information about both the current market state and its existing allocation. Therefore, the complete state vector has 50 dimensions and can be expressed as

$$\mathbf{s}_t = [\mathbf{x}_{1,t}, \mathbf{x}_{2,t}, \dots, \mathbf{x}_{5,t}, \mathbf{w}_t], \quad (1)$$

where $\mathbf{x}_{i,t} \in \mathbb{R}^9$ denotes the feature vector of asset i at time t and $\mathbf{w}_t \in \mathbb{R}^5$ represents the current portfolio weights. Hence,

$$5 \times 9 + 5 = 50. \quad (2)$$

Including the current portfolio weights allows the agent to account for the difference between its existing allocation and the target allocation generated at the current decision step.

C. Chronological Data Partitioning

The dataset is divided chronologically into training, validation, and testing periods to prevent future observations from being used during model development. The training period extends from March 16, 2015, to December 31, 2021, and contains 1,714 trading days. The validation period covers January 1, 2022, to December 31, 2023, with 501 trading days. The final test period covers January 1, 2024, to December 31, 2024, with 252 trading days.

The 2024 period is treated as a held-out out-of-sample period and is not used for model training or validation. The trained policy is evaluated on this period after model development is completed. This chronological arrangement preserves the temporal ordering of financial observations and

TABLE I
DDPG TRAINING CONFIGURATION

Parameter	Value
State dimension	50
Action dimension	5
Initial portfolio value	\$100,000
Initial allocation	20% per asset
Training episodes	15
Replay buffer size	100,000
Batch size	64
Actor learning rate	1×10^{-4}
Critic learning rate	1×10^{-3}
Discount factor (γ)	0.99
Target update rate (τ)	0.005
Exploration noise (σ)	0.10
Transaction-cost rate	0.001

provides a separate period for assessing the learned allocation policy.

D. Experimental Configuration

The portfolio environment starts with an initial capital of \$100,000 and equal 20% allocation across the five assets. At each decision step, the DDPG Actor produces five continuous allocation values corresponding to the five assets. The resulting allocation is normalized so that the portfolio remains fully invested and satisfies the long-only constraint.

A transaction-cost rate of 0.001 is applied to portfolio turnover. Let $\mathbf{w}_{t-1}$ denote the previous portfolio weights and $\mathbf{w}_t$ the newly selected weights. The portfolio turnover is calculated as

$$T_t = \sum_{i=1}^5 |w_{i,t} - w_{i,t-1}|. \quad (3)$$

The transaction cost is then determined by

$$C_t = cT_t, \quad (4)$$

where $c = 0.001$ is the transaction-cost coefficient. The net portfolio reward accounts for the portfolio return after this transaction cost.

The DDPG agent is trained using 15 episodes with experience replay and target networks. Gaussian exploration noise with a standard deviation of 0.10 is applied during training. The main training configuration is summarized in Table I.

IV. PROPOSED METHODOLOGY

A. Overall Framework

The proposed framework formulates adaptive portfolio allocation as a sequential decision-making problem. At each time step, the portfolio environment provides the DDPG agent with the current market state and portfolio allocation. The Actor network processes this state and generates a continuous allocation action for the five assets. The selected allocation is then applied to the portfolio environment, where the resulting portfolio return and transaction cost are calculated.

The overall workflow of the proposed framework is illustrated in Fig. ?? . Historical market data are first processed to

obtain the technical features for each asset. These features are combined with the current portfolio weights to form the 50-dimensional state provided to the DDPG agent. The Actor generates the target portfolio allocation, while the Critic evaluates the quality of the state-action pair. The portfolio environment applies the allocation, calculates the resulting reward, and provides the next market state. During training, the resulting transitions are stored in the replay buffer and used to update the Actor and Critic networks.

The interaction between the agent and the portfolio environment can be represented as

$$\mathbf{s}_t \rightarrow \mathbf{a}_t \rightarrow r_t \rightarrow \mathbf{s}_{t+1}, \quad (5)$$

where $\mathbf{s}_t$ denotes the current state, $\mathbf{a}_t$ represents the allocation action generated by the Actor, r_t is the resulting reward, and $\mathbf{s}_{t+1}$ denotes the subsequent state.

B. Feature Engineering

Nine market features are generated for each asset to represent price movement, trend, momentum, and price position relative to Bollinger Bands. The feature set consists of daily return, 20-day simple moving average ratio, 50-day simple moving average ratio, normalized 14-day Relative Strength Index (RSI), normalized Moving Average Convergence Divergence (MACD), normalized MACD signal, normalized MACD histogram, Bollinger Band position, and Bollinger Band width.

The daily return is calculated as

$$R_t = \frac{P_t - P_{t-1}}{P_{t-1}}, \quad (6)$$

where P_t denotes the closing price at time t .

The moving-average ratios are calculated using the 20-day and 50-day simple moving averages:

$$SMARatio_{n,t} = \frac{P_t}{SMA_{n,t}} - 1, \quad n \in \{20, 50\}. \quad (7)$$

The 14-day RSI is normalized as

$$RSI_t^* = \frac{RSI_{14,t}}{100}. \quad (8)$$

The MACD, its signal line, and the corresponding histogram are normalized by the closing price:

$$MACD_t^* = \frac{MACD_t}{P_t}, \quad Signal_t^* = \frac{Signal_t}{P_t}, \quad (9)$$

and

$$Hist_t^* = \frac{MACD_t - Signal_t}{P_t}. \quad (10)$$

The Bollinger Band position is calculated as

$$BBPos_t = \frac{P_t - BB_t^{low}}{BB_t^{high} - BB_t^{low} + 10^{-8}}, \quad (11)$$

while the Bollinger Band width is represented as

$$BBWidth_t = \frac{BB_t^{high} - BB_t^{low}}{100}. \quad (12)$$

The nine features are calculated independently for each asset and concatenated to form the 45-dimensional market component of the state.

C. Portfolio State and Action

The state provided to the agent combines the market representation with the current portfolio allocation. Let $\mathbf{x}_{i,t}$ denote the nine-dimensional feature vector associated with asset i at time t . The market component is represented as

$$\mathbf{x}_t = [\mathbf{x}_{1,t}, \mathbf{x}_{2,t}, \dots, \mathbf{x}_{5,t}], \quad (13)$$

where $\mathbf{x}_t \in \mathbb{R}^{45}$. The current portfolio weights are then appended to obtain the complete state:

$$\mathbf{s}_t = [\mathbf{x}_t, \mathbf{w}_t] \in \mathbb{R}^{50}. \quad (14)$$

Thus, the state consists of 45 market features and five portfolio-weight features.

The Actor maps the observed state to a continuous action:

$$\mathbf{a}_t = \pi_\theta(\mathbf{s}_t), \quad (15)$$

where π_θ denotes the Actor parameterized by θ . The five action values correspond to the target allocations for AAPL, MSFT, GOOGL, AMZN, and NVDA.

The resulting allocation is normalized by the portfolio environment to satisfy the long-only fully invested constraint:

$$w_{i,t} \geq 0, \quad \sum_{i=1}^5 w_{i,t} = 1. \quad (16)$$

Therefore, the action represents a continuous portfolio allocation rather than a discrete buy, sell, or hold decision.

D. Reward and Transaction Cost

The reward used by the portfolio environment is based on the net portfolio return after accounting for the transaction cost associated with changing the portfolio allocation. Let R_t^p denote the portfolio return before transaction costs and C_t denote the corresponding transaction cost. The reward is defined as

$$r_t = R_t^p - C_t. \quad (17)$$

Transaction costs are determined from the turnover between the previous and newly selected portfolio allocations. Let $\mathbf{w}_{t-1}$ represent the previous portfolio weights and $\mathbf{w}_t$ represent the newly selected weights. Portfolio turnover is calculated as

$$T_t = \sum_{i=1}^5 |w_{i,t} - w_{i,t-1}|. \quad (18)$$

Using the transaction-cost coefficient $c = 0.001$, the corresponding cost is

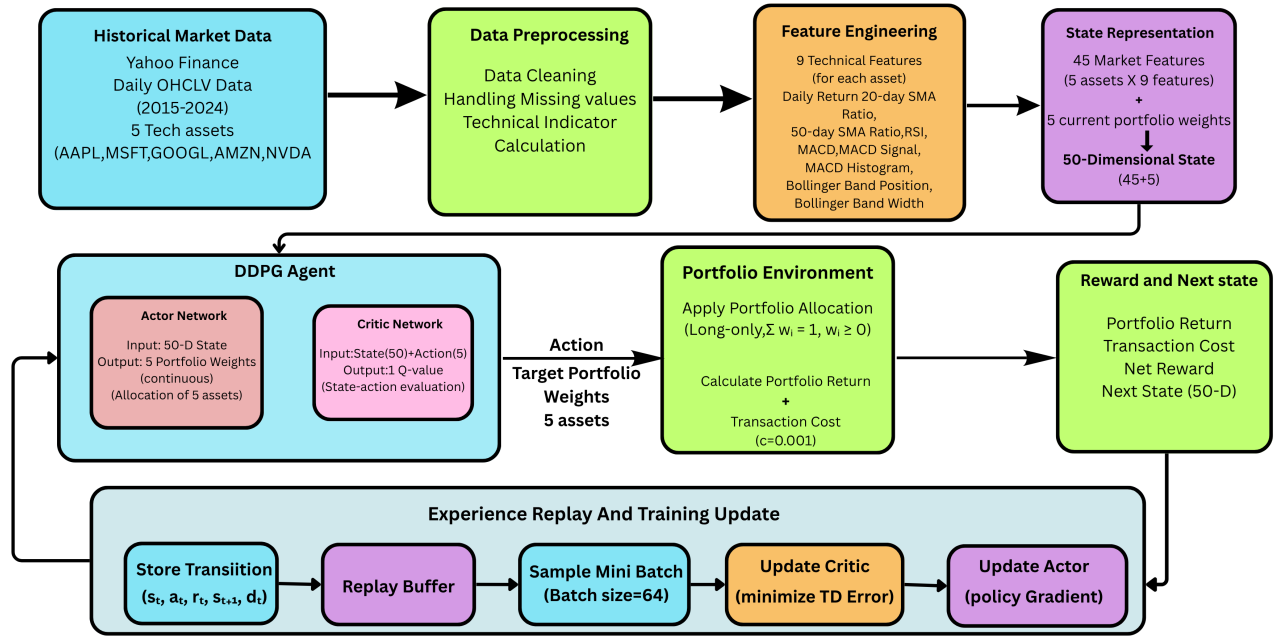


Fig. 1. Overall workflow of the proposed DDPG-based adaptive portfolio allocation framework.

$$C_t = cT_t. \quad (19)$$

Consequently, changes in portfolio allocation affect the reward through the associated transaction cost. The formulation therefore evaluates the allocation using its net portfolio outcome rather than considering gross portfolio return alone.

E. Portfolio Environment and Sequential Decision Process

The portfolio environment manages the interaction between the learned policy and the historical market data. At each decision step, the environment receives the allocation generated by the Actor and applies the corresponding portfolio weights to the five assets. The portfolio return is then determined from the asset returns and the selected allocation, after which the transaction cost is deducted to obtain the net reward.

Following the execution of an allocation, the environment updates the portfolio state and advances to the next available market observation. The updated portfolio weights become part of the next state provided to the agent. This allows subsequent allocation decisions to depend not only on the latest market features but also on the current portfolio composition. The sequential interaction can therefore be expressed as

$$(s_t, \mathbf{a}_t) \rightarrow (R_t^p, C_t, r_t, s_{t+1}), \quad (20)$$

where the environment converts the selected action into a portfolio outcome and returns the corresponding reward and next state.

During training, each transition is stored in the experience replay buffer in the form

$$(s_t, \mathbf{a}_t, r_t, s_{t+1}, d_t), \quad (21)$$

where d_t indicates whether the episode has reached its terminal state. Mini-batches sampled from the replay buffer are subsequently used to update the DDPG Actor and Critic networks.

V. DDPG ARCHITECTURE AND TRAINING

A. Actor Network

The Actor network represents the policy used to generate the portfolio allocation from the observed market state. The input to the Actor is the 50-dimensional state vector described in Section IV. The network consists of two fully connected hidden layers with 256 neurons each, with ReLU activation applied after each hidden layer. The final fully connected layer produces five outputs corresponding to the five assets in the portfolio.

The Actor therefore represents the policy mapping

$$\mathbf{a}_t = \pi_\theta(s_t), \quad (22)$$

where $s_t \in \mathbb{R}^{50}$ denotes the current state, $\mathbf{a}_t \in \mathbb{R}^5$ denotes the continuous action, and θ represents the Actor parameters. The five output values correspond to the target allocations for AAPL, MSFT, GOOGL, AMZN, and NVDA. The resulting action is processed by the portfolio environment to satisfy the long-only fully invested portfolio constraint.

The Actor is updated using the policy gradient obtained from the Critic. This allows the policy to increase the estimated value of the actions generated for the observed portfolio states.

B. Critic Network

The Critic network evaluates the quality of the action generated by the Actor for a given portfolio state. Unlike

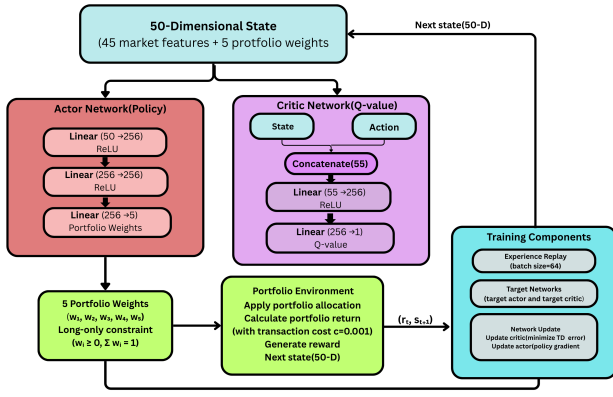


Fig. 2. Architecture of the DDPG Actor-Critic networks and training components.

the Actor, which generates the portfolio allocation, the Critic estimates the expected value of a state-action pair using a scalar Q-value.

The Critic receives the 50-dimensional state and the five-dimensional portfolio action. The state is first processed through its state pathway before being combined with the action representation. The resulting joint representation is then processed through fully connected layers to produce a single Q-value.

The Critic function is represented as

$$Q_{\phi}(\mathbf{s}_t, \mathbf{a}_t), \quad (23)$$

where ϕ denotes the Critic parameters. The estimated Q-value provides the learning signal used to improve the Actor policy and to train the Critic toward more accurate value estimates.

The roles of the two networks are therefore distinct: the Actor determines the portfolio allocation, whereas the Critic evaluates the quality of that allocation for the observed market state.

C. Experience Replay and Target Networks

During training, the interaction between the DDPG agent and the portfolio environment produces transitions of the form

$$(\mathbf{s}_t, \mathbf{a}_t, r_t, \mathbf{s}_{t+1}, d_t), \quad (24)$$

where d_t indicates whether the current episode has reached a terminal state. These transitions are stored in a replay buffer with a maximum capacity of 100,000 transitions.

Instead of updating the networks using only the most recent transition, mini-batches of 64 transitions are sampled from the replay buffer. Experience replay reduces the dependence between consecutive observations and allows previously collected transitions to be reused during training.

DDPG also maintains target versions of the Actor and Critic networks. For a sampled transition, the target Q-value is calculated using the target networks as

$$y_t = r_t + \gamma(1 - d_t)Q_{\phi'}(\mathbf{s}_{t+1}, \pi_{\theta'}(\mathbf{s}_{t+1})), \quad (25)$$

where γ is the discount factor, θ' represents the target Actor parameters, and ϕ' represents the target Critic parameters.

The Critic is trained by minimizing the mean squared temporal-difference error:

$$L(\phi) = \frac{1}{B} \sum_{i=1}^B (Q_{\phi}(\mathbf{s}_i, \mathbf{a}_i) - y_i)^2, \quad (26)$$

where B denotes the mini-batch size.

The Actor is updated using the gradient of the Critic with respect to the actions generated by the Actor. Its objective can be written as

$$J(\theta) = \frac{1}{B} \sum_{i=1}^B Q_{\phi}(\mathbf{s}_i, \pi_{\theta}(\mathbf{s}_i)), \quad (27)$$

with the Actor parameters updated in the direction that increases the estimated Q-value.

The target networks are updated using soft updates:

$$\theta' \leftarrow \tau\theta + (1 - \tau)\theta', \quad (28)$$

$$\phi' \leftarrow \tau\phi + (1 - \tau)\phi', \quad (29)$$

where $\tau = 0.005$ is the target-network update rate.

D. DDPG Training Procedure

The training process begins with the initial portfolio state and uses the Actor to generate a continuous allocation for the five assets. During training, Gaussian exploration noise with a standard deviation of 0.10 is applied to encourage exploration of alternative allocation actions. The selected allocation is passed to the portfolio environment, which calculates the portfolio return, transaction cost, reward, and subsequent state.

The resulting transition is stored in the replay buffer. Once training updates are performed, mini-batches are sampled from the stored transitions and used to update the Critic and Actor networks. The target Actor and target Critic are subsequently updated using the soft-update equations.

The complete training cycle can therefore be summarized as

$$\mathbf{s}_t \rightarrow \pi_{\theta}(\mathbf{s}_t) \rightarrow \mathbf{a}_t \rightarrow (r_t, \mathbf{s}_{t+1}) \rightarrow \mathcal{B} \rightarrow \text{Critic Update} \rightarrow \text{Actor Update}, \quad (30)$$

where $\mathcal{B}$ denotes the replay buffer.

The DDPG agent is trained for 15 episodes using the configuration presented in Table I. Model development uses the chronological training and validation periods defined in Section III. The model corresponding to the selected validation performance is retained for subsequent evaluation on the held-out 2024 test period. The test period is not used for parameter updates or model selection.

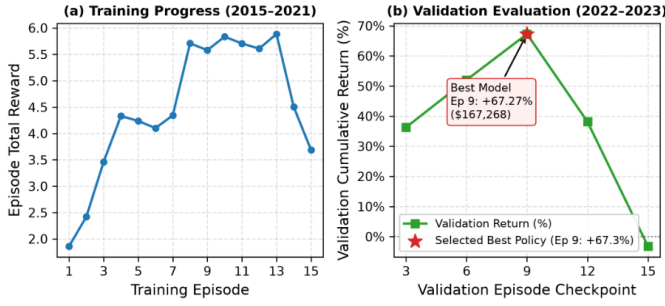


Fig. 3. Training reward and validation performance across DDPG training episodes.

VI. RESULTS AND DISCUSSION

A. Training and Validation Performance

The DDPG policy was trained over 15 complete episodes across the 1,714 trading days of the training split (March 16, 2015 to December 31, 2021), with deterministic validation evaluations executed at three-episode intervals on the independent 501-day validation split (January 1, 2022 to December 31, 2023). Fig. 3 illustrates the training progress and validation trajectory across episodes.

As shown in Fig. 3(a), the training episode reward increased from 1.861 in Episode 1 to a peak of 5.886 in Episode 13, before declining to 4.503 and 3.688 in Episodes 14 and 15, respectively. The training reward did not exhibit strictly monotonic improvement. In the validation evaluation shown in Fig. 3(b), deterministic portfolio returns increased from +36.28% in Episode 3 (\$136,280.89) and +52.01% in Episode 6 (\$152,006.39) to a maximum cumulative return of +67.27% in Episode 9, corresponding to a final validation portfolio value of \$167,268.04 and a validation Sharpe ratio of 0.76. Subsequent validation evaluations recorded lower returns of +38.14% in Episode 12 and -3.20% in Episode 15. Based on validation cumulative return, the checkpoint from Episode 9 was selected for evaluation on the held-out test period.

B. Out-of-Sample Test Results

The selected Episode 9 DDPG policy was evaluated on the strictly held-out 2024 test period (252 trading days) alongside an Equal-Weight benchmark. The empirical results are reported in Table II, and the corresponding daily portfolio trajectories are plotted in Fig. 4.

Starting from an initial allocation of \$100,000, the Equal-Weight benchmark achieved a final portfolio value of \$157,948.88 (+57.95% total return), whereas the DDPG policy finished at \$144,625.75 (+44.63% total return). The DDPG strategy recorded an annualized return of 44.84% with an annualized volatility of 38.41%, yielding an annualized Sharpe ratio of 1.17 and a maximum drawdown of 22.08%. By contrast, the Equal-Weight benchmark recorded an annualized return of 58.24% with an annualized volatility of 22.39%, resulting in a Sharpe ratio of 2.60 and a maximum drawdown of 17.31%.

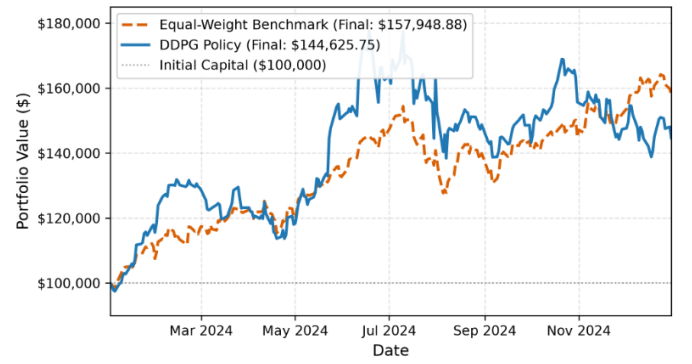


Fig. 4. Portfolio value trajectories of the DDPG policy and Equal-Weight benchmark during the held-out 2024 test period.

During the first half of 2024, the DDPG policy experienced rapid capital growth, reaching an intra-year peak of \$177,569.49 on July 10, 2024, at which point the Equal-Weight benchmark stood at \$149,944.82. Between mid-July and early August 2024, the DDPG portfolio subsequently declined to \$138,367.63 on August 7, corresponding to the observed maximum drawdown of 22.08%. A secondary peak occurred on November 11, 2024 at \$169,139.79 before the portfolio closed the year at \$144,625.75. The DDPG agent accumulated \$12,780.30 in total transaction costs, compared with \$344.62 incurred by the Equal-Weight benchmark.

C. Dynamic Portfolio Allocation

Fig. 5 displays the daily portfolio weight allocations generated by the trained DDPG policy across the five constituent assets throughout 2024. The policy satisfied the continuous long-only and fully invested constraints ($\sum_{i=1}^5 w_i = 1.0$, $w_i \geq 0$) at each decision step through Softmax normalization.

The allocation history reveals substantial state-dependent concentration rather than balanced diversification. Across the 252 test days, NVIDIA (NVDA) received the largest average allocation, with a mean weight of 59.69%, exceeding 50% on 149 days and 90% on 129 days. Apple (AAPL) was the second most weighted asset, with a mean weight of 22.51%, exceeding 50% allocation on 58 days and 90% on 34 days. Alphabet (GOOGL) received an average allocation of 9.65% and exceeded 50% allocation on 24 days, while Amazon (AMZN) averaged 6.22% and exceeded 50% allocation on 9 days. Microsoft (MSFT) received the lowest average allocation of 1.92% and never exceeded 39.86% on any single trading day.

These allocation patterns indicate frequent shifts in portfolio exposure toward individual equities. The concentration coincided with periods of portfolio growth during increases in NVDA and AAPL prices, while also exposing the portfolio to greater single-stock fluctuations. The active allocation changes were accompanied by substantially higher transaction costs, with the DDPG portfolio incurring \$12,780.30 over the test period.

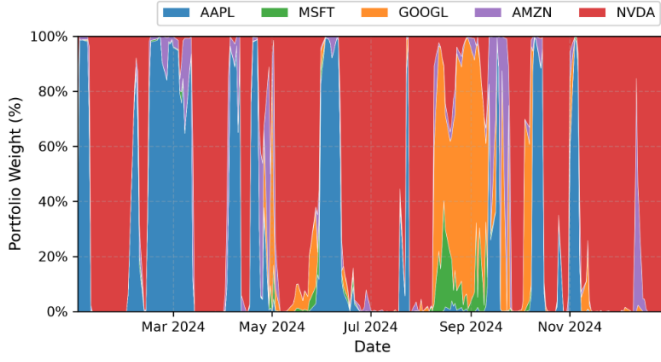


Fig. 5. Daily portfolio allocation generated by the DDPG policy during the 2024 test period.

D. Discussion

The experimental results show that the trained DDPG policy generated continuous, state-dependent portfolio allocation vectors using the defined market features and current portfolio weights. Rather than maintaining a fixed allocation throughout the test period, the policy changed asset exposure in response to the observed state.

The held-out 2024 evaluation also highlights several characteristics of the resulting portfolio:

- 1) **Benchmark Performance:** In the 2024 test period, the Equal-Weight benchmark achieved a higher cumulative return (+57.95% vs. +44.63%) and a higher Sharpe ratio (2.60 vs. 1.17), while exhibiting lower annualized volatility (22.39% vs. 38.41%) and a lower maximum drawdown (17.31% vs. 22.08%). Under the evaluated test period, the DDPG policy therefore did not exceed the Equal-Weight benchmark on these performance measures.
- 2) **Transaction Cost Friction:** The continuous allocation policy incurred substantially higher transaction costs. Under the proportional transaction-cost coefficient of $c = 0.001$, the DDPG portfolio accumulated \$12,780.30 in transaction costs, equivalent to 12.78% of the initial \$100,000 capital, compared with \$344.62 for the Equal-Weight benchmark.
- 3) **Concentration and Volatility:** The learned allocation pattern frequently concentrated exposure in NVDA and AAPL, with allocations exceeding 90% observed for both assets during portions of the test period. This concentration coincided with periods of strong portfolio growth as well as larger fluctuations in portfolio value, including the observed 22.08% maximum drawdown.

VII. CONCLUSION

This work presented a DDPG-based framework for continuous portfolio allocation across five U.S. technology equities using a 50-dimensional market and portfolio state. The policy was evaluated on the held-out 2024 test period with transaction costs. The DDPG portfolio achieved a cumulative return of

TABLE II
HELD-OUT 2024 OUT-OF-SAMPLE PERFORMANCE

Metric	DDPG	Equal Weight
Final Value	\$144,625.75	\$157,948.88
Cumulative Return	44.63%	57.95%
Annualized Return	44.84%	58.24%
Annualized Volatility	38.41%	22.39%
Sharpe Ratio	1.17	2.60
Maximum Drawdown	22.08%	17.31%
Transaction Costs	\$12,780.30	\$344.62

44.63%, compared with 57.95% for the Equal-Weight benchmark, while also recording higher volatility, drawdown, and transaction costs. The allocation analysis showed substantial concentration in NVDA and AAPL. These results provide an empirical assessment of continuous reinforcement-learning-based portfolio allocation and highlight the importance of transaction costs and concentration in evaluating adaptive portfolio policies.

REFERENCES

- [1] Z. Jiang, D. Xu, and J. Liang, "A deep reinforcement learning framework for the financial portfolio management problem," arXiv preprint arXiv:1706.10059, 2017.
- [2] X.-Y. Liu, Z. Xiong, S. Zhong, H. Yang, and A. Walid, "Practical deep reinforcement learning approach for stock trading," arXiv preprint arXiv:1811.07522, 2018.
- [3] H. Yang, X.-Y. Liu, S. Zhong, and A. Walid, "Deep reinforcement learning for automated stock trading: An ensemble strategy," in *Proc. 2nd ACM Int. Conf. AI in Finance (ICAIF)*, 2020, doi: 10.1145/3383455.3422540.
- [4] C. Betancourt and W.-H. Chen, "Deep reinforcement learning for portfolio management of markets with a dynamic number of assets," *Expert Systems with Applications*, vol. 164, Art. no. 114002, 2021, doi: 10.1016/j.eswa.2020.114002.
- [5] Z. Shen and H. Li, "Portfolio management based on deep reinforcement learning method with data augment," *Procedia Computer Science*, vol. 221, pp. 1122–1130, 2023, doi: 10.1016/j.procs.2023.08.097.
- [6] J. Jang and N. Seong, "Deep reinforcement learning for stock portfolio optimization by connecting with modern portfolio theory," *Expert Systems with Applications*, vol. 218, Art. no. 119556, 2023, doi: 10.1016/j.eswa.2023.119556.
- [7] Y. Jiang, J. Olmo, and M. Atwi, "Deep reinforcement learning for portfolio selection," *Global Finance Journal*, vol. 62, Art. no. 101016, 2024, doi: 10.1016/j.gfj.2024.101016.
- [8] L.-C. Cheng and J.-S. Sun, "Multiagent-based deep reinforcement learning framework for multi-asset adaptive trading and portfolio management," *Neurocomputing*, vol. 594, Art. no. 127800, 2024, doi: 10.1016/j.neucom.2024.127800.
- [9] M. Jiang, Z. Xu, and Z. Lin, "Dynamic risk control and asset allocation using Q-learning in financial markets," *Transactions on Computational and Scientific Methods*, vol. 4, no. 12, 2024, doi: 10.5281/zenodo.14499791.
- [10] H. Choudhary, A. Orra, K. Sahoo, and M. Thakur, "Risk-adjusted deep reinforcement learning for portfolio optimization: A multi-reward approach," *International Journal of Computational Intelligence Systems*, vol. 18, no. 1, Art. no. 126, 2025, doi: 10.1007/s44196-025-00875-8.